# Assignment 8: Basic SQL Queries and Aggregate Functions

**Data Retrieval, Conditional Filtering, Multi-Level Sorting & Statistical Aggregation**

**Student Name:** Sanjeev Adarsh S
**Program / Semester:** BCA • Semester III
**Course:** Database Management Systems (DBMS Lab)
**Deliverable:** Individual Assignment 8 (PDF)

## 1. Introduction to SQL Data Retrieval Mechanics

Structured Query Language (SQL) data manipulation capabilities allow database administrators and application services to extract actionable intelligence from normalized relational stores. SQL query processing follows a strict logical execution sequence distinct from its written lexical syntax:

```
Logical Query Execution Order:
1. FROM / JOIN     → Identifies source relations and evaluates Cartesian products/join predicates
2. WHERE           → Filters individual candidate tuples based on boolean predicates
3. GROUP BY        → Partitions surviving tuples into aggregated clusters sharing identical keys
4. HAVING          → Discards aggregated clusters that fail group-level summary predicates
5. SELECT          → Evaluates projections, expressions, scalar functions, and column aliases
6. ORDER BY        → Sorts the finalized result set chronologically or alphabetically
7. LIMIT / OFFSET  → Restricts the number of transmitted output rows
```

This assignment demonstrates five core SQL query paradigms executed directly against the populated **Smart Library Management System (SLMS)**: conditional record filtering ( `WHERE` ), hierarchical record sorting ( `ORDER BY` ), categorical grouping ( `GROUP BY` ), group-level filtering ( `HAVING` ), and foundational statistical aggregate functions ( `COUNT` , `SUM` , `AVG` , `MIN` , `MAX` ). Each query includes the executable SQL statement, verbatim tabular terminal output, and a concise technical explanation.

## 2. Detailed Query Demonstrations & Technical Explanations

### 2.1 Query 1: Filtering Records Using Compound `WHERE` Predicates

**Operational Objective:** Identify high-value academic textbooks with significant remaining shelf availability for reserve reading desk placement. The query extracts books with a retail price exceeding $50.00 and at least 3 copies currently available in circulation.

```
-- Query 1: Compound WHERE clause with relational and logical operators
SELECT book_id, title, publication_year, price, available_copies
FROM books
WHERE price > 50.00 AND available_copies >= 3;
```

```
+-----------+---------------------------------------+-------------------+---------+--------------------+
|   book_id | title                                 |   publication_year |   price |   available_copies |
|-----------+---------------------------------------+-------------------+---------+--------------------|
|         1 | Introduction to Algorithms (4th Ed)   |              2022 |   89.99 |                  5 |
|         4 | Database System Concepts (7th Ed)     |              2019 |   85.00 |                  3 |
|         6 | The Art of Computer Programming, Vol 1 |              1997 |   75.00 |                  4 |
+-----------+---------------------------------------+-------------------+---------+--------------------+
(3 rows selected)
```

**Technical Explanation:** The query evaluates the relational selection condition `(price > 50.00) AND (available_copies >= 3)` across all rows in `books` . Out of 10 catalog editions, only 3 meet both criteria. Titles like *"Artificial Intelligence: A Modern Approach"* ($99.50) are filtered out because their available copy count (2) fails the secondary predicate.

### 2.2 Query 2: Multi-Column Hierarchical Sorting Using `ORDER BY`

**Operational Objective:** Generate an active patron roster organized by membership classification tier (alphabetically ascending) and ordered chronologically by join date (most recently admitted members first).

```
-- Query 2: Multi-level ORDER BY with ASC and DESC directives
SELECT member_id, full_name, membership_type, join_date, max_books_allowed
FROM members
WHERE status = 'Active'
ORDER BY membership_type ASC, join_date DESC;
```

```
+-------------+--------------------+------------------+-------------+--------------------+
|   member_id | full_name          | membership_type  | join_date   |   max_books_allowed |
|-------------+--------------------+------------------+-------------+--------------------|
|           7 | Dhananjay Shankar  | Alumni           | 2023-05-18  |                  3 |
|           6 | Prof. Renita Joy   | Faculty          | 2024-06-20  |                 10 |
|           5 | Dr. Arisudan Sharma | Faculty          | 2024-01-15  |                 10 |
|           4 | Mahesh M.          | Student          | 2025-08-12  |                  5 |
|           3 | Vigneshvar K.K.    | Student          | 2025-08-10  |                  5 |
|           2 | Hemanth Krishna    | Student          | 2025-08-05  |                  5 |
|           1 | Sanjeev Adarsh     | Student          | 2025-08-01  |                  5 |
+-------------+--------------------+------------------+-------------+--------------------+
(7 rows selected)
```

**Technical Explanation:** The `WHERE status = 'Active'` predicate eliminates suspended patron Kabila Varshini. The surviving tuples are sorted first by `membership_type ASC` (Alumni → Faculty → Student). Within each category partition, tuples are sorted secondarily by `join_date DESC`, ensuring newer student admissions appear before earlier ones.

### 2.3 Query 3: Categorical Grouping Using `GROUP BY` with Aggregations

**Operational Objective:** Analyze demographic distributions across patron tiers, computing the total registered headcount and average borrowing entitlement quota per membership group.

```
-- Query 3: GROUP BY categorical grouping with COUNT and AVG
SELECT membership_type, COUNT(*) AS member_count, AVG(max_books_allowed) AS avg_quota
FROM members
GROUP BY membership_type;
```

```
+------------------+---------------+-------------+
| membership_type  |   member_count |   avg_quota |
|------------------+---------------+-------------|
| Alumni           |             1 |         3.0 |
| Faculty          |             2 |        10.0 |
| Guest            |             1 |         2.0 |
| Student          |             4 |         5.0 |
+------------------+---------------+-------------+
(4 rows selected)
```

**Technical Explanation:** The `GROUP BY membership_type` clause collapses the 8 member tuples into 4 distinct equivalence classes. The aggregate function `COUNT(*)` computes the tuple cardinality of each group, while `AVG(max_books_allowed)` calculates the mean quota, accurately reflecting the tiered institutional policy (Faculty: 10 books, Students: 5 books, Alumni: 3 books, Guests: 2 books).

### 2.4 Query 4: Group Filtering Using `HAVING` Clause

**Operational Objective:** Identify well-represented academic curriculum disciplines possessing multiple textbook titles, calculating the aggregate physical volume units stocked within each qualifying category.

```
-- Query 4: Filtering aggregated group clusters using HAVING
SELECT category_id, COUNT(*) AS books_in_category, SUM(total_copies) AS sum_inventory
FROM books
GROUP BY category_id
HAVING COUNT(*) > 1;
```

```
+--------------+--------------------+-----------------+
|   category_id |   books_in_category |   sum_inventory |
|--------------+--------------------+-----------------|
|            1 |                  4 |              25 |
|            3 |                  2 |              15 |
|            4 |                  2 |              15 |
+--------------+--------------------+-----------------+
(3 rows selected)
```

**Technical Explanation:** The query groups `books` by `category_id`. The critical distinction between `WHERE` and `HAVING` is demonstrated: while `WHERE` filters individual input rows before grouping, `HAVING COUNT(*) > 1` evaluates aggregate group properties, discarding Category 2 (Database Systems - 1 title) and Category 6 (Information Systems - 1 title). Category 1 (Computer Science & AI) qualifies with 4 distinct titles representing 25 physical book copies.

### 2.5 Query 5: Comprehensive Statistical Aggregate Functions

**Operational Objective:** Generate an executive catalog summary report computing the total number of registered titles, total physical inventory copies in circulation/shelves, average textbook retail price, minimum price, and maximum price across the institution.

```
-- Query 5: Utilizing COUNT(), SUM(), AVG(), MIN(), and MAX() in a single projection
SELECT
    COUNT(*) AS total_book_titles,
    SUM(total_copies) AS total_inventory_units,
    ROUND(AVG(price), 2) AS average_book_price,
    MIN(price) AS lowest_price,
    MAX(price) AS highest_price
FROM books;
```

```
+--------------------+-------------------------+---------------------+----------------+-----------------+
|   total_book_titles |   total_inventory_units |   average_book_price |   lowest_price |   highest_price |
|--------------------+-------------------------+---------------------+----------------+-----------------|
|                 10 |                      67 |               63.09 |          11.95 |           99.50 |
+--------------------+-------------------------+---------------------+----------------+-----------------+
(1 row selected)
```

**Technical Explanation:** This query applies all five standard ANSI aggregate functions across the entire relation:

- `COUNT(*)`: Evaluates total catalog titles registered (\(N = 10\)).
- `SUM(total_copies)`: Aggregates total physical copies across all shelves (\(67\) units).
- `ROUND(AVG(price), 2)`: Computes the arithmetic mean unit price (\(\$63.09\)), rounded to currency precision.
- `MIN(price)`: Identifies the lowest catalog acquisition price (\(\$11.95\) for *"Information: A Very Short Introduction"*).
- `MAX(price)`: Identifies the highest catalog acquisition price (\(\$99.50\) for *"Artificial Intelligence: A Modern Approach"*).

## 3. Summary & Administrative Value

These basic SQL queries demonstrate the declarative power of relational databases. By combining relational selection (`WHERE`), multi-level sorting (`ORDER BY`), equivalence partitioning (`GROUP BY`), post-aggregation filtering (`HAVING`), and mathematical aggregations, library administrators obtain immediate real-time visibility into asset distribution, circulation limits, and institutional collection value.

## 4. Technical Deep-Dive: Aggregation Semantics & Query Performance

Executing aggregations and sorting operations at enterprise scale requires a clear understanding of database engine mechanics:

- **`COUNT(*)` vs. `COUNT(column_name)` Semantics:** While `COUNT(*)` computes the raw cardinality of tuples in an equivalence class (including rows where optional attributes might be NULL), `COUNT(column_name)` evaluates only non-NULL occurrences. In financial ledgers like `fines`, computing `COUNT(payment_date)` accurately isolates settled penalties from pending uncollected fines.
- **Index-Driven Aggregation & Sort Acceleration:** Without secondary indexes, an `ORDER BY` operation forces the query execution engine to materialize results into an in-memory or on-disk sort work-area (external merge sort). By establishing a composite B-Tree index on `members(membership_type, join_date)`, the engine performs a direct presorted index scan, eliminating CPU sort overhead entirely.
- **Two-Pass Grouping vs Hash Aggregation:** Modern relational engines implement grouping via either pre-sorted streams or in-memory hash tables. For low-cardinality categorical fields like `membership_type` (4 unique values) and `category_id` (6 unique values), hash aggregation achieves near-instantaneous \(O(N)\) linear processing times.

> ***Practical Takeaway:*** *The queries execute deterministically against the BCNF-normalized schema without requiring temporary staging tables or complex application-side loop filtering. Indexing foreign keys (such as `category_id`) and search attributes (such as `price` and `status`) ensures that these aggregation queries execute with optimal algorithmic complexity (\(O(\log N)\) index scans) even as the university catalog scales to hundreds of thousands of volumes.*